Automated Lemma Synthesis in Symbolic-Heap Separation Logic

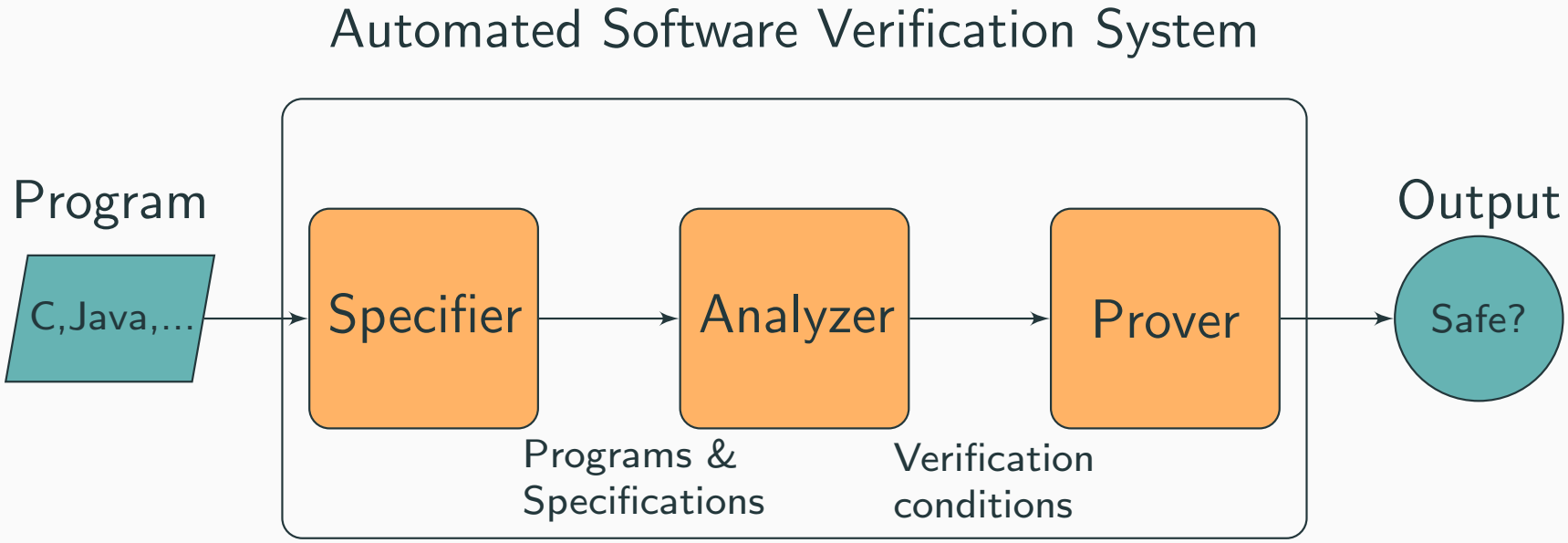
Presenter: Quang-Trung Ta

in collaboration with

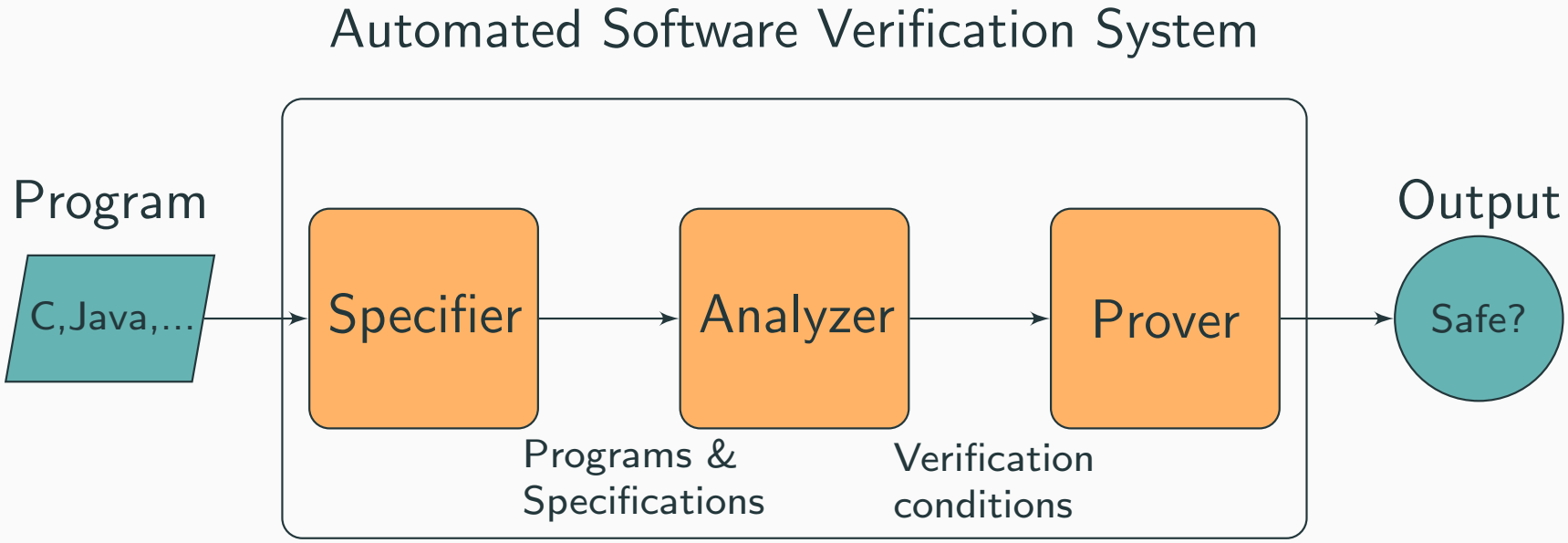
Ton-Chanh Le, Siau-Cheng Khoo, Wei-Ngan Chin

National University of Singapore, Singapore

Automated Software Verification by Using Separation Logic

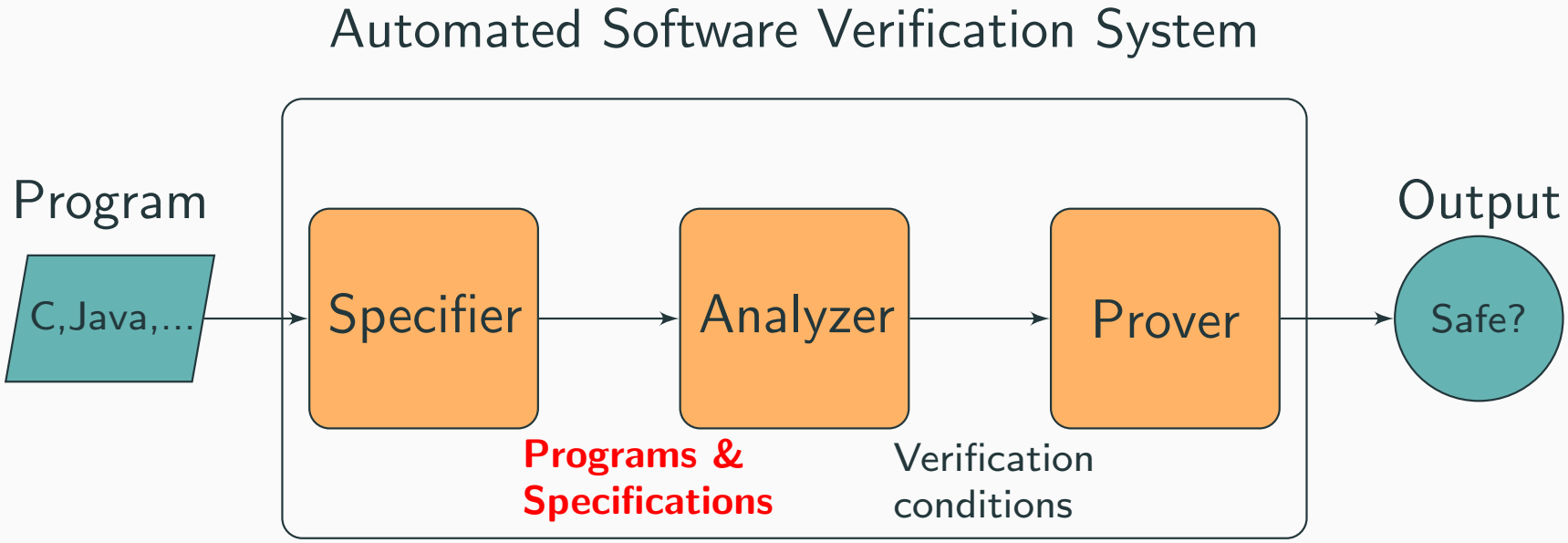


Automated Software Verification by Using Separation Logic



Verify a program in 3 main steps:

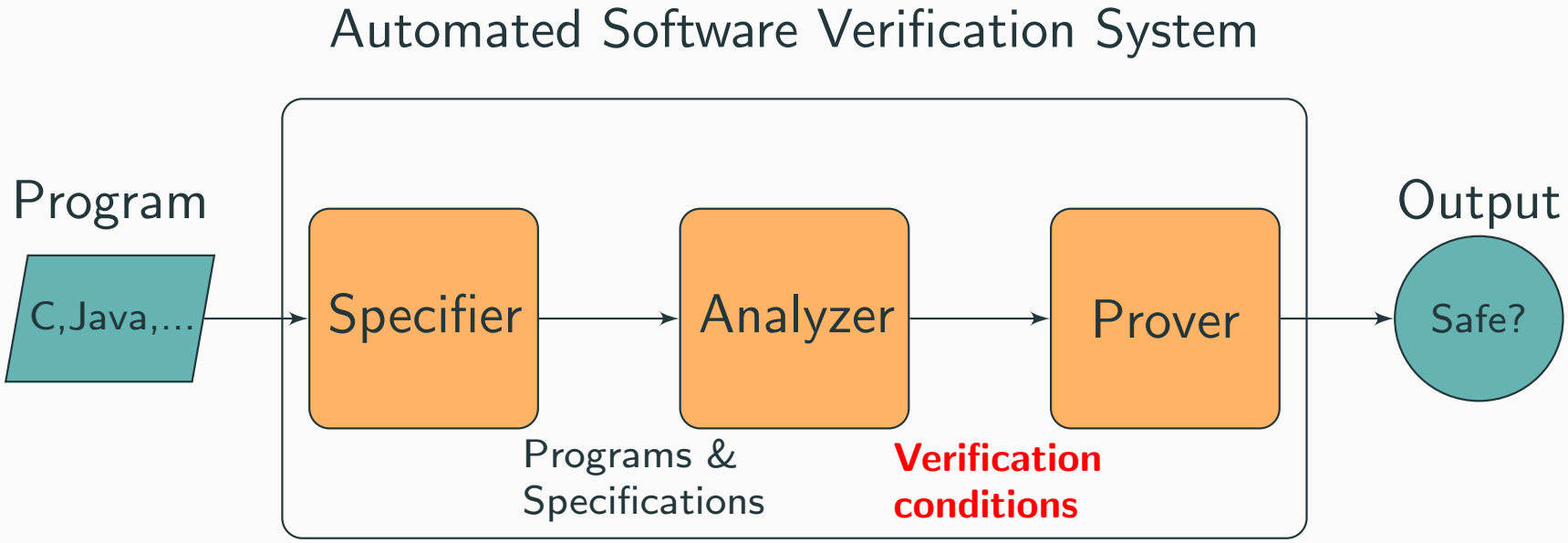
Automated Software Verification by Using Separation Logic



Verify a program in 3 main steps:

1. Specify the program behaviors using *separation logic formulas*.

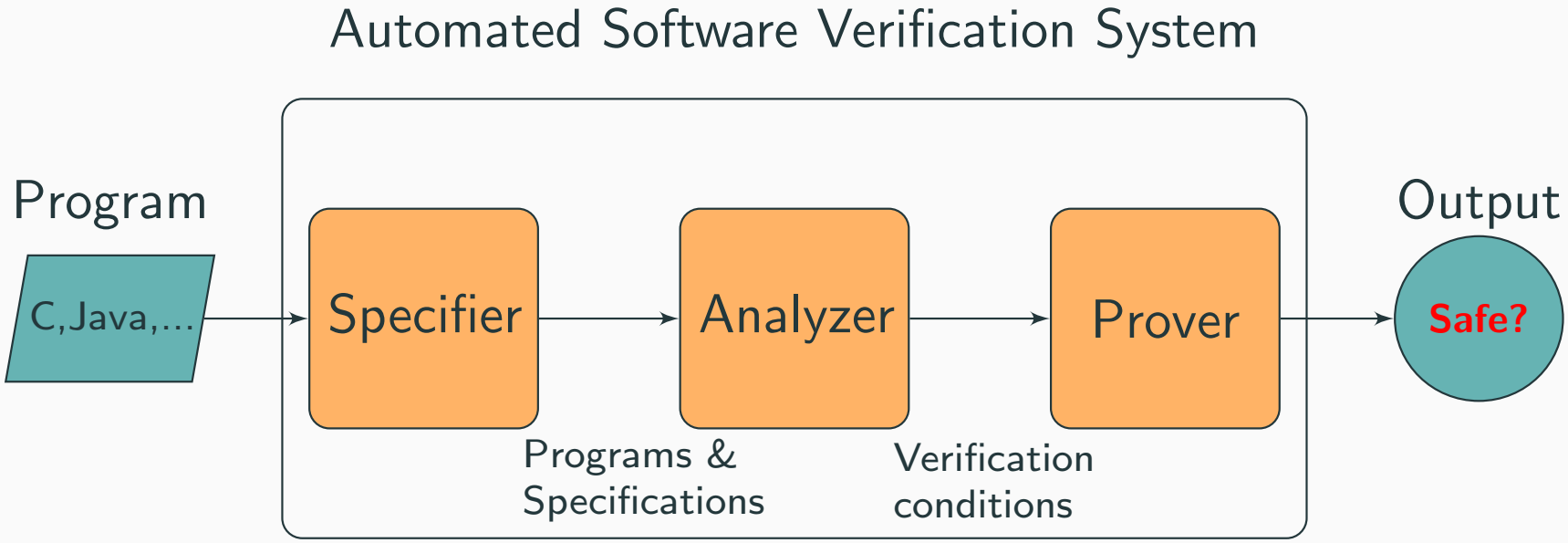
Automated Software Verification by Using Separation Logic



Verify a program in 3 main steps:

1. Specify the program behaviors using *separation logic formulas*.
2. Analyze the program against its specifications.

Automated Software Verification by Using Separation Logic



Verify a program in 3 main steps:

1. Specify the program behaviors using *separation logic formulas*.
2. Analyze the program against its specifications.
3. Prove the verification conditions to decide the program's safety.

Why Using Separation Logic?

- Provide an expressive model of the *content* and the *separation* of data structures stored in computer memory:
  - Example: a singly-linked-list data structure `cell`

```
struct cell {
    int value;
    struct cell* next;
};
```

  - Model the content of each element: $x \mapsto 1, \mathtt{nil}$.
  - Model the separation of elements: $y \mapsto 2, z \ * \ z \mapsto 3, \mathtt{nil}$.

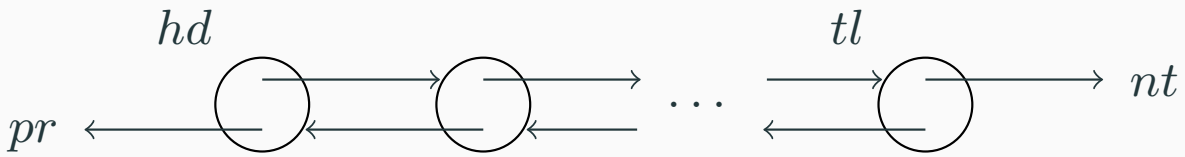
Why Using Separation Logic?

- Model data structures' shapes using *inductive heap predicates*:
  - Example: segments of a doubly-linked-list data structure `node`

```
struct node {
    struct node* prev;
    struct node* next;
};
```

$$\mathsf{dll}(hd, pr, tl, nt, len) \stackrel{\text{def}}{=} hd{\mapsto}pr, nt \wedge hd{=}tl \wedge len{=}1 \;\vee\; \exists u.(hd{\mapsto}pr, u * \mathsf{dll}(u, hd, tl, nt, len{-}1))$$

$$\mathsf{dllrev}(hd, pr, tl, nt, len) \stackrel{\text{def}}{=} hd{\mapsto}pr, nt \wedge hd{=}tl \wedge len{=}1 \;\vee\; \exists u.(\mathsf{dllrev}(hd, pr, u, tl, len{-}1) * tl{\mapsto}u, nt)$$



What Are the Verification Conditions?

- They are constraints generated when:
  - Proving a *pre-condition* of a function call.
  - Proving a *post-condition* of a function definition.
  - Normalizing or transforming a program state.

What Are the Verification Conditions?

- They are constraints generated when:
  - Proving a *pre-condition* of a function call.
  - Proving a *post-condition* of a function definition.
  - Normalizing or transforming a program state.

- They appear in the form of separation logic *entailments*:
  - $\mathsf{dll}(x, y, z, t, n) \vdash \mathsf{dllrev}(x, y, z, t, n)$
  - $\mathsf{dll}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, m) \vdash \mathsf{dll}(x, y, z, t, n{+}m)$

Proving Separation Logic Entailments

- Popular approaches: *pre-define syntax and semantics* of heap predicates [Berdine et al., FSTTCS'2004, Cook et al., CONCUR'2011, Pérez and Rybalchenko, PLDI'2011], or *translate separation logic formulas* into formulas in graph, automata, set theories [Iosif et al., CADE'2013, Piskac et al., CAV'2013, Enea et al., APLAS'2014].

Proving Separation Logic Entailments

- Popular approaches: *pre-define syntax and semantics* of heap predicates [Berdine et al., FSTTCS'2004, Cook et al., CONCUR'2011, Pérez and Rybalchenko, PLDI'2011], or *translate separation logic formulas* into formulas in graph, automata, set theories [Iosif et al., CADE'2013, Piskac et al., CAV'2013, Enea et al., APLAS'2014].
  - Weakness: restrict the expressiveness of heap predicates.

Proving Separation Logic Entailments

- Popular approaches: *pre-define syntax and semantics* of heap predicates [Berdine et al., FSTTCS'2004, Cook et al., CONCUR'2011, Pérez and Rybalchenko, PLDI'2011], or *translate separation logic formulas* into formulas in graph, automata, set theories [Iosif et al., CADE'2013, Piskac et al., CAV'2013, Enea et al., APLAS'2014].
  - Weakness: restrict the expressiveness of heap predicates.

- Cyclic proof [Brotherston et al., CADE'2011] and induction proof [Chu et al., PLDI'2015, Ta et al., FM'2016] approaches: *apply induction hypotheses* to prevent the unlimited unfolding of inductive heap predicates.

Proving Separation Logic Entailments

- Popular approaches: *pre-define syntax and semantics* of heap predicates [Berdine et al., FSTTCS'2004, Cook et al., CONCUR'2011, Pérez and Rybalchenko, PLDI'2011], or *translate separation logic formulas* into formulas in graph, automata, set theories [Iosif et al., CADE'2013, Piskac et al., CAV'2013, Enea et al., APLAS'2014].
  - Weakness: restrict the expressiveness of heap predicates.

- Cyclic proof [Brotherston et al., CADE'2011] and induction proof [Chu et al., PLDI'2015, Ta et al., FM'2016] approaches: *apply induction hypotheses* to prevent the unlimited unfolding of inductive heap predicates.
  - Weakness: they are *incomplete* proof techniques, due to the failure of *cut-elimination* in inductive theories [Bundy, JAR'2001].

Motivating Example

- Prove the following entailment E:

$$E : \mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

Motivating Example

- Prove the following entailment E:

$$E : \mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

- Approaches that pre-define predicates or translate formulas: *do not support both* arithmetic constraints (length) and reverse-fashion predicates (dllrev)
  [Berdine et al., FSTTCS'2004, Cook et al., CONCUR'2011, Pérez and Rybalchenko, PLDI'2011, Iosif et al., CADE'2013, Piskac et al., CAV'2013, Enea et al., APLAS'2014].

Attempt to Prove E by Structural Induction...

- Original entailment:

  E : $\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

Attempt to Prove E by Structural Induction...

- Original entailment:

  E : $\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
- Record it as an induction hypothesis:

  IH : $\mathsf{dllrev}(a,b,p,q,m) * \mathsf{dll}(q, p, c, d, 200) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$

Attempt to Prove E by Structural Induction...

- Original entailment:

  E : $\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
- Record it as an induction hypothesis:

  IH : $\mathsf{dllrev}(a,b,p,q,m) * \mathsf{dll}(q, p, c, d, 200) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$
- Unfold $\mathsf{dllrev}(x, y, u, v, n)$ of E by inductive case:

  E_1' : $\mathsf{dllrev}(x, y, w, u, n{-}1) * u \mapsto v, w \ * \ \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

Attempt to Prove E by Structural Induction...

- Original entailment:

  E : $\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
- Record it as an induction hypothesis:

  IH : $\mathsf{dllrev}(a, b, p, \mathbf{q}, m) * \mathsf{dll}(\mathbf{q}, p, c, d, 200) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$
- Unfold $\mathsf{dllrev}(x, y, u, v, n)$ of E by inductive case:

  E_1' : $\mathsf{dllrev}(x, y, w, \mathbf{u}, n{-}1) * u \mapsto v, w \ * \ \mathsf{dll}(\mathbf{v}, u, z, t, 200) \wedge n \geq 100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

  $\Rightarrow$ Cannot apply IH to prove E_1': $\mathbf{q}$ is mapped to $\mathbf{u}$ and $\mathbf{v}$ when unifying dllrev, dll.

Attempt to Prove E by Structural Induction...

- Original entailment:

  E : $\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
- Record it as an induction hypothesis:

  IH : $\mathsf{dllrev}(a,b,p,q,m) * \mathsf{dll}(q, p, c, d, 200) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$
- Unfold $\mathsf{dllrev}(x, y, u, v, n)$ of E by inductive case:

  E_1' : $\mathsf{dllrev}(x, y, w, u, n{-}1) * u \mapsto v, w \ * \ \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

  $\Rightarrow$ Cannot apply IH to prove E_1': q is mapped to u and v when unifying dllrev, dll.
- Fold $u \mapsto v, w * \mathsf{dll}(v, u, z, t, 200)$ of E_1' into $\mathsf{dll}(u, w, z, t, 201)$:

  E_1'' : $\mathsf{dllrev}(x, y, w, u, n{-}1) * \mathsf{dll}(u, w, z, t, 201) \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

Attempt to Prove E by Structural Induction...

- Original entailment:

  E : $\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
- Record it as an induction hypothesis:

  IH : $\mathsf{dllrev}(a,b,p,q,m) * \mathsf{dll}(q, p, c, d, \mathbf{200}) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$
- Unfold $\mathsf{dllrev}(x, y, u, v, n)$ of E by inductive case:

  E_1' : $\mathsf{dllrev}(x, y, w, u, n{-}1) * u \mapsto v, w \ * \ \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

  $\Rightarrow$ Cannot apply IH to prove E_1': q is mapped to u and v when unifying dllrev, dll.
- Fold $u \mapsto v, w * \mathsf{dll}(v, u, z, t, 200)$ of E_1' into $\mathsf{dll}(u, w, z, t, 201)$:

  E_1'' : $\mathsf{dllrev}(x, y, w, u, n{-}1) * \mathsf{dll}(u, w, z, t, \mathbf{201}) \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$

  $\Rightarrow$ Cannot apply IH to prove E_1'': $\mathbf{200}$ is mapped to $\mathbf{201}$ when unifying dll

Attempt to Prove E by Structural Induction...

- Original entailment:
  E : dllrev$(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)$
- Record it as an induction hypothesis:
  IH : dllrev$(a,b,p,q,m) * \mathsf{dll}(q, p, c, d, 200) \wedge m{\geq}100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c{\mapsto}k, d)$
- Unfold dllrev(x, y, u, v, n) of E by inductive case:
  E_1' : dllrev$(x, y, w, u, n{-}1) * u{\mapsto}v, w \ * \ \mathsf{dll}(v, u, z, t, 200) \wedge n{\geq}100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)$
  $\Rightarrow$ Cannot apply IH to prove E_1': q is mapped to u and v when unifying dllrev, dll.
- Fold $u{\mapsto}v, w * \mathsf{dll}(v, u, z, t, 200)$ of E_1' into $\mathsf{dll}(u, w, z, t, 201)$:
  E_1'' : dllrev$(x, y, w, u, n{-}1) * \mathsf{dll}(u, w, z, t, 201) \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)$
  $\Rightarrow$ Cannot apply IH to prove E_1'': 200 is mapped to 201 when unifying dll

- Structural induction proof fails to prove E when unfolding dllrev(x, y, u, v, n).

Attempt to Prove E by Structural Induction...

- Original entailment:
  E : $\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
- Record it as an induction hypothesis:
  IH : $\mathsf{dllrev}(a,b,p,q,m) * \mathsf{dll}(q, p, c, d, 200) \wedge m \geq 100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c \mapsto k, d)$
- Unfold $\mathsf{dllrev}(x, y, u, v, n)$ of E by inductive case:
  E_1' : $\mathsf{dllrev}(x, y, w, u, n{-}1) * u \mapsto v, w \ * \ \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
  $\Rightarrow$ Cannot apply IH to prove E_1': q is mapped to u and v when unifying dllrev, dll.
- Fold $u \mapsto v, w * \mathsf{dll}(v, u, z, t, 200)$ of E_1' into $\mathsf{dll}(u, w, z, t, 201)$:
  E_1'' : $\mathsf{dllrev}(x, y, w, u, n{-}1) * \mathsf{dll}(u, w, z, t, 201) \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$
  $\Rightarrow$ Cannot apply IH to prove E_1'': 200 is mapped to 201 when unifying dll

- Structural induction proof fails to prove E when unfolding $\mathsf{dllrev}(x, y, u, v, n)$.
- It also fails when unfolding $\mathsf{dll}(v, u, z, t, 200)$.

Attempt to Prove E by Structural Induction...

- Original entailment:
  E : $\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)$
- Record it as an induction hypothesis:
  IH : $\mathsf{dllrev}(a,b,p,q,m) * \mathsf{dll}(q, p, c, d, 200) \wedge m{\geq}100 \vdash \exists k.(\mathsf{dll}(a, b, k, c, n{+}199) * c{\mapsto}k, d)$
- Unfold $\mathsf{dllrev}(x, y, u, v, n)$ of E by inductive case:
  E_1' : $\mathsf{dllrev}(x, y, w, u, n{-}1) * u{\mapsto}v, w \ * \ \mathsf{dll}(v, u, z, t, 200) \wedge n{\geq}100 \vdash$
  $\exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)$
  $\Rightarrow$ Cannot apply IH to prove E_1': q is mapped to u and v when unifying dllrev, dll.
- Fold $u{\mapsto}v, w * \mathsf{dll}(v, u, z, t, 200)$ of E_1' into $\mathsf{dll}(u, w, z, t, 201)$:
  E_1'' : $\mathsf{dllrev}(x, y, w, u, n{-}1) * \mathsf{dll}(u, w, z, t, 201) \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z{\mapsto}r, t)$
  $\Rightarrow$ Cannot apply IH to prove E_1'': 200 is mapped to 201 when unifying dll

- Structural induction proof fails to prove E when unfolding $\mathsf{dllrev}(x, y, u, v, n)$.
- It also fails when unfolding $\mathsf{dll}(v, u, z, t, 200)$.
- In general, existing cyclic and induction proofs fail to prove E directly [Brotherston et al., CADE'2011, Chu et al., PLDI'2015, Ta et al., FM'2016].

However, E Can Be Proved by Using Auxiliary Lemmas...

- Original entailment:

$$\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

- Original entailment:

$$\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

$$\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

- Original entailment:

$$\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

$$\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

$$\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

$$L_1 \triangleq \mathsf{dllrev}(a, b, c, d, m) \vdash \mathsf{dll}(a, b, c, d, m)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

$$\frac{\mathsf{dll}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)}{\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)} \; \mathsf{LM_L} \; L_1$$

$$L_1 \triangleq \mathsf{dllrev}(a, b, c, d, m) \vdash \mathsf{dll}(a, b, c, d, m)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

$$\frac{\mathsf{dll}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)}{\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)}\ \mathsf{LM}_{\mathrm{L}}\ L_1$$

$$L_1 \triangleq \mathsf{dllrev}(a, b, c, d, m) \vdash \mathsf{dll}(a, b, c, d, m)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)$$

$$\frac{\mathsf{dll}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)}{\mathsf{dllrev}(x, y, u, v, n) * \mathsf{dll}(v, u, z, t, 200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x, y, r, z, n{+}199) * z \mapsto r, t)}\ \mathsf{LM}_{\mathrm{L}}\ L_1$$

$$L_1 \triangleq \mathsf{dllrev}(a, b, c, d, m) \vdash \mathsf{dll}(a, b, c, d, m)$$
$$L_2 \triangleq \mathsf{dll}(a, b, p, q, m) * \mathsf{dll}(q, p, c, d, l) \vdash \mathsf{dll}(a, b, c, d, m + l)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z \mapsto r, t)$$

$$\dfrac{\dfrac{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z \mapsto r, t)}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z \mapsto r, t)}\ \mathsf{LM_L}\ L_2}{\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z \mapsto r, t)}\ \mathsf{LM_L}\ L_1$$

$$L_1 \triangleq \mathsf{dllrev}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_2}{\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_1$$

$$L_1 \triangleq \mathsf{dllrev}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_2}{\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_1$$

$$L_1 \triangleq \mathsf{dllrev}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$
$$L_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_R}\ L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_2}{\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_1$$

$$L_1 \triangleq \mathsf{dllrev}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$
$$L_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_R}\ L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_2}{\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_1$$

$$\begin{aligned}
L_1 &\triangleq \mathsf{dllrev}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)\\
L_2 &\triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)\\
L_3 &\triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)
\end{aligned}$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\mathsf{P}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_R}\ L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_2}{\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_1$$

$$L_1 \triangleq \mathsf{dllrev}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$
$$L_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\mathsf{P}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_R}\ L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_2}{\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_1$$

$$L_1 \triangleq \mathsf{dllrev}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$
$$L_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\ \vdash_{\Pi}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}\ *\mathsf{P}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_R}\ L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_2}{\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\ \mathsf{LM_L}\ L_1$$

$$L_1 \triangleq \mathsf{dllrev}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$
$$L_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)$$

However, E Can Be Proved by Using Auxiliary Lemmas...

▷ Original entailment:

$$\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathsf{true} \vdash x{=}x \wedge y{=}y \wedge z{=}z \wedge t{=}t \wedge n{+}200{=}n{+}200}\vdash_{\Pi}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \mathsf{dll}(x,y,z,t,n{+}200)}*\mathsf{P}}{\mathsf{dll}(x,y,z,t,n{+}200) \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\mathsf{LM_R}\ L_3}{\mathsf{dll}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\mathsf{LM_L}\ L_2}{\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n{\geq}100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n{+}199) * z{\mapsto}r,t)}\mathsf{LM_L}\ L_1$$

$$L_1 \triangleq \mathsf{dllrev}(a,b,c,d,m) \vdash \mathsf{dll}(a,b,c,d,m)$$
$$L_2 \triangleq \mathsf{dll}(a,b,p,q,m) * \mathsf{dll}(q,p,c,d,l) \vdash \mathsf{dll}(a,b,c,d,m+l)$$
$$L_3 \triangleq \mathsf{dll}(a,b,c,d,m) \wedge m{\geq}2 \vdash \exists w.(\mathsf{dll}(a,b,w,c,m{-}1) * c{\mapsto}w,d)$$

▷ **How to synthesize the auxiliary lemmas?**

Our Lemma Synthesis Approach

Synthesize lemmas to assist in proving a goal entailment in 2 steps:

Our Lemma Synthesis Approach

Synthesize lemmas to assist in proving a goal entailment in 2 steps:

1. Discover lemma templates: synthesize the lemmas' *heap structures*

Our Lemma Synthesis Approach

Synthesize lemmas to assist in proving a goal entailment in 2 steps:

1. Discover lemma templates: synthesize the lemmas' *heap structures*
2. Refine the lemma templates: synthesize *Boolean constraints* over the lemma templates' variables:
   - Synthesize constraints in the *antecedents*
   - Synthesize constraints in the *consequents*

Step 1: Discover the Lemma Templates

- **Idea**: choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied)

Step 1: Discover the Lemma Templates

- **Idea**: choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied)
- Example: the motivating entailment E:
  $\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$

Step 1: Discover the Lemma Templates

- **Idea**: choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied)
- Example: the motivating entailment E:
  $\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$
- Possible lemma templates:

Step 1: Discover the Lemma Templates

- **Idea**: choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied)
- Example: the motivating entailment E:
  $\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$
- Possible lemma templates:
  $\mathsf{dllrev}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,x_6,x_7,x_8,n_2.\mathsf{dll}(x_5,x_6,x_7,x_8,n_2)$

Step 1: Discover the Lemma Templates

- **Idea**: choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied)

- Example: the motivating entailment E:
  $\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$

- Possible lemma templates:
  $\mathsf{dllrev}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,x_6,x_7,x_8,n_2.\mathsf{dll}(x_5,x_6,x_7,x_8,n_2)$
  $\mathsf{dll}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,\ldots,x_{11},n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9 \mapsto x_{10},x_{11})$

Step 1: Discover the Lemma Templates

- **Idea**: choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied)

- Example: the motivating entailment E:
  $\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$

- Possible lemma templates:
  $\mathsf{dllrev}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,x_6,x_7,x_8,n_2.\mathsf{dll}(x_5,x_6,x_7,x_8,n_2)$
  $\mathsf{dll}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,\ldots,x_{11},n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9 \mapsto x_{10},x_{11})$
  $\mathsf{dllrev}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,\ldots,n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9 \mapsto x_{10},x_{11})$

Step 1: Discover the Lemma Templates

- **Idea**: choose templates having similar heap structures with the goal entailment (so that the lemmas can be applied)

- Example: the motivating entailment E:
  $\mathsf{dllrev}(x,y,u,v,n) * \mathsf{dll}(v,u,z,t,200) \wedge n \geq 100 \vdash \exists r.(\mathsf{dll}(x,y,r,z,n+199) * z \mapsto t,r)$

- Possible lemma templates:
  $\mathsf{dllrev}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,x_6,x_7,x_8,n_2.\mathsf{dll}(x_5,x_6,x_7,x_8,n_2)$
  $\mathsf{dll}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,\ldots,x_{11},n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9 \mapsto x_{10},x_{11})$
  $\mathsf{dllrev}(x_1,x_2,x_3,x_4,n_1) \vdash \exists x_5,\ldots,n_2.(\mathsf{dll}(x_5,x_6,x_7,x_8,n_2) * x_9 \mapsto x_{10},x_{11})$
  $\mathsf{dllrev}(x_1,x_2,x_3,x_4,n_1) * \mathsf{dll}(x_5,x_6,x_7,x_8,n_2) \vdash \exists x_9,\ldots,n_3.\mathsf{dll}(x_9,x_{10},x_{11},x_{12},n_3)$

Step 2: Refine the Lemma Templates

- Refine the simplest lemma templates first.

  $T_1 \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

  $T_2 \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$

Step 2: Refine the Lemma Templates

- Refine the simplest lemma templates first.
  $T_1 \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
  $T_2 \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$

- **Idea**: synthesize Boolean constraints over all variables x_i, n_i in each template (pointer and integer variables).

Step 2: Refine the Lemma Templates

- Refine the simplest lemma templates first.
  $T_1 \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
  $T_2 \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$
- **Idea**: synthesize Boolean constraints over all variables x_i, n_i in each template (pointer and integer variables).
- Where to place the Boolean constraints?

Step 2: Refine the Lemma Templates

- Refine the simplest lemma templates first.
  $T_1 \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
  $T_2 \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$
- **Idea**: synthesize Boolean constraints over all variables x_i, n_i in each template (pointer and integer variables).
- Where to place the Boolean constraints?
  - In the antecedents:
    $\mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge ? \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge ? \vdash \exists x_5, \ldots, x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$

Step 2: Refine the Lemma Templates

- Refine the simplest lemma templates first.
  $T_1 \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
  $T_2 \triangleq \mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$
- **Idea**: synthesize Boolean constraints over all variables x_i, n_i in each template (pointer and integer variables).
- Where to place the Boolean constraints?
  - In the antecedents:
    $\mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge ? \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge ? \vdash \exists x_5, \ldots, x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$
  - In the consequents:
    $\mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, x_6, x_7, x_8, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) \wedge ?)$
    $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, \ldots, x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Goal: synthesize variable constraints in the antecedent:

(T_1) : $\mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge ? \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Goal: synthesize variable constraints in the antecedent:

  (T_1) : $\mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge \textbf{?} \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

- **Idea**:
  - Represent the desired constraint by an *unknown relation* $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2)$ (denoted as $U(\vec{x}, \vec{n})$).

Step 2.1: Refine the Lemma Templates' Antecedents

- Goal: synthesize variable constraints in the antecedent:

  (T_1) : $\mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge \mathbf{?} \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

- **Idea**:
  - Represent the desired constraint by an *unknown relation* $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2)$ (denoted as $U(\vec{x}, \vec{n})$).
  - Create an unknown entailment:

    $E_{u_1} \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge U(\vec{x}, \vec{n}) \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Goal: synthesize variable constraints in the antecedent:
  (T_1) : $\mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge \textbf{?} \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

- **Idea**:
  - Represent the desired constraint by an *unknown relation* $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2)$ (denoted as $U(\vec{x}, \vec{n})$).
  - Create an unknown entailment:
    $E_{u_1} \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge U(\vec{x}, \vec{n}) \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
  - *Prove* the unknown entailment E_{u_1} by induction and *collect* a constraint set of $U(\vec{x}, \vec{n})$.

Step 2.1: Refine the Lemma Templates' Antecedents

- Goal: synthesize variable constraints in the antecedent:
  (T_1) : $\mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge \mathbf{?} \vdash \exists x_5, x_6, x_7, x_8, n_2.\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

- **Idea**:
  - Represent the desired constraint by an *unknown relation* $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2)$ (denoted as $U(\vec{x}, \vec{n})$).
  - Create an unknown entailment:
    $E_{u_1} \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge U(\vec{x}, \vec{n}) \vdash \mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$
  - *Prove* the unknown entailment E_{u_1} by induction and *collect* a constraint set of $U(\vec{x}, \vec{n})$.
  - *Solve* the constraint set to find out the actual definition of $U(\vec{x}, \vec{n})$ and discover the lemma.

Step 2.1: Refine the Lemma Templates' Antecedents

▷ Prove E_{u_1} by induction:

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathcal{A}_1 \triangleq \{x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1\}}{x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1}\,\mathsf{U}^1_\Pi}{x_1{\mapsto}x_2, x_4 \wedge x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash x_5{\mapsto}x_6, x_8 \wedge x_5{=}x_7 \wedge n_2{=}1}\,{*}{\mapsto}}{x_1{\mapsto}x_2, x_4 \wedge x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,\mathsf{P_R} \quad \dfrac{\dfrac{\dfrac{\mathcal{A}_2 \triangleq \{U(\vec{x},\vec{n}) \rightarrow \mathsf{false}\}}{x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash a_5{=}x_5 \wedge a_6{=}x_6 \wedge a_7{=}x_7 \wedge a_8{=}x_8 \wedge b_2{=}n_2}\,\mathsf{U}^1_\Sigma}{\mathsf{dll}(a_5,a_6,a_7,a_8,b_2) * x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,{*}\mathsf{P} \quad \mathcal{A}_3 \triangleq \{U(\vec{x},\vec{n}) \rightarrow U(\vec{a},\vec{b})\theta\}}{\mathsf{dllrev}(x_1,x_2,u,x_3,n_1{-}1) * x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,\mathsf{U_{IH}}}{E_{u_1} \triangleq \mathsf{dllrev}(x_1,x_2,x_3,x_4,n_1) \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,\mathsf{ID}$$

Where:

- $U(\vec{x},\vec{n}) \equiv U(x_1,x_2,x_3,x_4,x_5,x_6,x_7,x_8,n_1,n_2)$
- $U(\vec{a},\vec{b}) \equiv U(a_1,a_2,a_3,a_4,a_5,a_6,a_7,a_8,b_1,b_2)$
- the rule ID records $H \triangleq \mathsf{dllrev}(a_1,a_2,a_3,a_4,b_1) \wedge U(\vec{a},\vec{b}) \vdash \mathsf{dll}(a_5,a_6,a_7,a_8,b_2)$
- the rule $\mathsf{U_{IH}}$ applies H with $\theta = [x_1/a_1, x_2/a_2, u/a_3, x_3/a_4, n_1{-}1/b_1]$

Step 2.1: Refine the Lemma Templates' Antecedents

▷ Prove E_{u_1} by induction:

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathcal{A}_1 \triangleq \{x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1\}}{x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1}\,\mathsf{U}^1_\Pi}{x_1{\mapsto}x_2, x_4 \wedge x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash x_5{\mapsto}x_6, x_8 \wedge x_5{=}x_7 \wedge n_2{=}1}\,{*}{\mapsto}}{x_1{\mapsto}x_2, x_4 \wedge x_1{=}x_3 \wedge n_1{=}1 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,\mathsf{P_R} \quad \dfrac{\dfrac{\dfrac{\mathcal{A}_2 \triangleq \{U(\vec{x},\vec{n}) \rightarrow \mathsf{false}\}}{x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash a_5{=}x_5 \wedge a_6{=}x_6 \wedge a_7{=}x_7 \wedge a_8{=}x_8 \wedge b_2{=}n_2}\,\mathsf{U}^1_\Sigma}{\mathsf{dll}(a_5,a_6,a_7,a_8,b_2) * x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,{*}\mathsf{P} \quad \mathcal{A}_3 \triangleq \{U(\vec{x},\vec{n}) \rightarrow U(\vec{a},\vec{b})\theta\}}{\mathsf{dllrev}(x_1,x_2,u,x_3,n_1{-}1) * x_3{\mapsto}u, x_4 \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,\mathsf{U_{IH}}}{E_{u_1} \triangleq \mathsf{dllrev}(x_1,x_2,x_3,x_4,n_1) \wedge U(\vec{x},\vec{n}) \vdash \mathsf{dll}(x_5,x_6,x_7,x_8,n_2)}\,\mathsf{ID}$$

Collect a constraint set $\mathcal{A} = \mathcal{A}_1 \cup \mathcal{A}_2 \cup \mathcal{A}_3$ over the unknown relation U:

1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1,x_2,x_3,x_4,x_5,x_6,x_7,x_8,n_1,n_2) \rightarrow x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
2. $U(x_1,x_2,x_3,x_4,x_5,x_6,x_7,x_8,n_1,n_2) \rightarrow \mathsf{false}$
3. $U(x_1,x_2,x_3,x_4,x_5,x_6,x_7,x_8,n_1,n_2) \rightarrow U(x_1,x_2,u,x_3,a_5,a_6,a_7,a_8,n_1{-}1,b_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
     $x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
     $x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
     $x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:
  - Create a relation template with unknown coefficients:

$$U(x_1, ..., x_m, n_1, ..., n_k) \triangleq \bigwedge_{i=1}^{M} (\sum_{j=1}^{m} c_j^i x_j + c_0^i \geq 0) \wedge \bigwedge_{i=1}^{K} (\sum_{j=1}^{n} d_j^i n_j + d_0^i \geq 0)$$

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
     $x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:
  - Create a relation template with unknown coefficients:
    $$U(x_1, ..., x_m, n_1, ..., n_k) \triangleq \bigwedge_{i=1}^{M} (\sum_{j=1}^{m} c_j^i x_j + c_0^i \geq 0) \wedge \bigwedge_{i=1}^{K} (\sum_{j=1}^{n} d_j^i n_j + d_0^i \geq 0)$$
  - Replace the relation template to the constraint set and apply Farkas' lemma to eliminate universal quantification over x_i, n_i.

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
     $x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:
  - Create a relation template with unknown coefficients:
    $$U(x_1, ..., x_m, n_1, ..., n_k) \triangleq \bigwedge_{i=1}^{M} (\sum_{j=1}^{m} c_j^i x_j + c_0^i \geq 0) \wedge \bigwedge_{i=1}^{K} (\sum_{j=1}^{n} d_j^i n_j + d_0^i \geq 0)$$
  - Replace the relation template to the constraint set and apply Farkas' lemma to eliminate universal quantification over x_i, n_i.
  - Use SMT solvers to find out the unknown coefficients c_j^i, d_j^i.

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
     $x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

- Use a template-based constraint solving technique based on Farkas lemma [Schrijver, 1986]:
  - Create a relation template with unknown coefficients:
    $$U(x_1, ..., x_m, n_1, ..., n_k) \triangleq \bigwedge_{i=1}^{M} (\sum_{j=1}^{m} c_j^i x_j + c_0^i \geq 0) \wedge \bigwedge_{i=1}^{K} (\sum_{j=1}^{n} d_j^i n_j + d_0^i \geq 0)$$
  - Replace the relation template to the constraint set and apply Farkas' lemma to eliminate universal quantification over x_i, n_i.
  - Use SMT solvers to find out the unknown coefficients c_j^i, d_j^i.
- Solve the *entire* set $\mathcal{A}$ to discover a *full solution*, or solve a *subset* of $\mathcal{A}$ to discover a *partial solution*.

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
     $x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
     $x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

- Solve the full set $\mathcal{A}$: discover only a spurious solution (not useful):

  $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \equiv$ false

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
     $x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

- Solve the full set $\mathcal{A}$: discover only a spurious solution (not useful):

  $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \equiv$ false

- Solve the subset containing (1):

  $\overline{U}(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \equiv x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2$

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
     $x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$ false
  3. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

- Solve the full set $\mathcal{A}$: discover only a spurious solution (not useful):

  $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \equiv$ false

- Solve the subset containing (1):

  $\overline{U}(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \equiv x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2$

  – Discover (and verify) the lemma $\overline{E}_{u_1}$:

  $\overline{E}_{u_1} \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2 \vdash$
  $\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

Step 2.1: Refine the Lemma Templates' Antecedents

- Solve the constraint set $\mathcal{A}$:
  1. $x_1{=}x_3 \wedge n_1{=}1 \wedge U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow$
     $x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_4{=}x_8 \wedge x_5{=}x_7 \wedge n_2{=}1$
  2. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow \mathsf{false}$
  3. $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \rightarrow U(x_1, x_2, u, x_3, a_5, a_6, a_7, a_8, n_1{-}1, b_2)$

- Solve the full set $\mathcal{A}$: discover only a spurious solution (not useful):

  $U(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \equiv \mathsf{false}$

- Solve the subset containing (1):

  $\overline{U}(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, n_1, n_2) \equiv x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2$

  – Discover (and verify) the lemma $\overline{E}_{u_1}$:

  $\overline{E}_{u_1} \triangleq \mathsf{dllrev}(x_1, x_2, x_3, x_4, n_1) \wedge x_1{=}x_5 \wedge x_2{=}x_6 \wedge x_3{=}x_7 \wedge x_4{=}x_8 \wedge n_1{=}n_2 \vdash$
  $\mathsf{dll}(x_5, x_6, x_7, x_8, n_2)$

  – Normalize to obtain the motivating lemma L_1:

  $L_1 \triangleq \mathsf{dllrev}(a, b, c, d, m) \vdash \mathsf{dll}(a, b, c, d, m)$

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:

(T_2) : $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge \textbf{?})$

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:

  (T_2) : $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge \textbf{?})$

- **Idea**:
  - Adding Boolean constraints on only the consequent maybe *not good enough* because the antecedent can be *too weak*.

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:

  (T_2) : $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$

- **Idea**:
  - Adding Boolean constraints on only the consequent maybe *not good enough* because the antecedent can be *too weak*.
  - **Phase 1: preprocess the lemma template:**

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:

  (T_2) : $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge \textbf{?})$

- **Idea**:
  - Adding Boolean constraints on only the consequent maybe *not good enough* because the antecedent can be *too weak*.
  - **Phase 1: preprocess the lemma template:**
    - Find *initial constraints* containing only the antecedent's variables to make the lemma template valid.

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:

  (T_2) : $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge \textbf{?})$

- **Idea**:
  - Adding Boolean constraints on only the consequent maybe *not good enough* because the antecedent can be *too weak*.
  - **Phase 1: preprocess the lemma template:**
    - Find *initial constraints* containing only the antecedent's variables to make the lemma template valid.
    - Solve the unknown entailment:

      $$\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge U_1(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,\ldots,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$$

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:

  (T_2) : $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$

- **Idea**:
  - Adding Boolean constraints on only the consequent maybe *not good enough* because the antecedent can be *too weak*.
  - **Phase 1: preprocess the lemma template:**
    - Find *initial constraints* containing only the antecedent's variables to make the lemma template valid.
    - Solve the unknown entailment:

      $$\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge U_1(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$$

    - Solution: $\overline{U}_1(x_1, x_2, x_3, x_4, n_1) \equiv n_1 \geq 2$

Step 2.2: Refine Lemma Templates' Consequents

- Goal: synthesize variable constraints in the consequent:

  (T_2) : $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$

- **Idea**:
  - Adding Boolean constraints on only the consequent maybe *not good enough* because the antecedent can be *too weak*.
  - **Phase 1: preprocess the lemma template:**
    - Find *initial constraints* containing only the antecedent's variables to make the lemma template valid.
    - Solve the unknown entailment:

      $$\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge U_1(x_1, x_2, x_3, x_4, n_1) \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$$

    - Solution: $\overline{U}_1(x_1, x_2, x_3, x_4, n_1) \equiv n_1 \geq 2$
    - Obtain the preprocessed lemma template:

      $$\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11})$$

Step 2.2: Refine Lemma Templates' Consequents

- The preprocessed lemma template:

$$\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash$$
$$\exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$$

Step 2.2: Refine Lemma Templates' Consequents

- The preprocessed lemma template:

$$\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$$

- **Idea**:
  - **Phase 2: fine-tune the preprocessed lemma template:**

Step 2.2: Refine Lemma Templates' Consequents

- The preprocessed lemma template:

  $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash$

  $\exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$

- **Idea**:
  - **Phase 2: fine-tune the preprocessed lemma template:**
    - $+$ Find sufficient constraints containing all variables.

Step 2.2: Refine Lemma Templates' Consequents

- The preprocessed lemma template:
  $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash$
  $\exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$
- **Idea**:
  - **Phase 2: fine-tune the preprocessed lemma template:**
    - Find sufficient constraints containing all variables.
    - Solve the unknown entailment:
      $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$
      $x_9 \mapsto x_{10}, x_{11} \wedge U_2(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_1, n_2))$

Step 2.2: Refine Lemma Templates' Consequents

- The preprocessed lemma template:
  $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash$
  $\exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$
- **Idea**:
  - **Phase 2: fine-tune the preprocessed lemma template:**
    - Find sufficient constraints containing all variables.
    - Solve the unknown entailment:
      $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$
      $x_9 \mapsto x_{10}, x_{11} \wedge U_2(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_1, n_2))$
    - Solution: $\overline{U}_2(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_1, n_2) \equiv$
      $x_1 = x_5 \wedge x_2 = x_6 \wedge x_3 = x_8 \wedge x_4 = x_{11} \wedge x_7 = x_{10} \wedge x_8 = x_9$

Step 2.2: Refine Lemma Templates' Consequents

- The preprocessed lemma template:
  $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash$
  $\exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$
- **Idea**:
  - **Phase 2: fine-tune the preprocessed lemma template:**
    - Find sufficient constraints containing all variables.
    - Solve the unknown entailment:
      $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$
      $x_9 \mapsto x_{10}, x_{11} \wedge U_2(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_1, n_2))$
    - Solution: $\overline{U}_2(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_1, n_2) \equiv$
      $x_1 = x_5 \wedge x_2 = x_6 \wedge x_3 = x_8 \wedge x_4 = x_{11} \wedge x_7 = x_{10} \wedge x_8 = x_9$
    - The discovered lemma:
      $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$
      $x_9 \mapsto x_{10}, x_{11} \wedge x_1 = x_5 \wedge x_2 = x_6 \wedge x_3 = x_8 \wedge x_4 = x_{11} \wedge x_7 = x_{10} \wedge x_8 = x_9)$

Step 2.2: Refine Lemma Templates' Consequents

- The preprocessed lemma template:
  $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash$
  $\exists x_5, ..., x_{11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) * x_9 \mapsto x_{10}, x_{11} \wedge ?)$
- **Idea**:
  - **Phase 2: fine-tune the preprocessed lemma template:**
    - Find sufficient constraints containing all variables.
    - Solve the unknown entailment:
      $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$
      $x_9 \mapsto x_{10}, x_{11} \wedge U_2(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_1, n_2))$
    - Solution: $\overline{U}_2(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}, x_{11}, n_1, n_2) \equiv$
      $x_1 = x_5 \wedge x_2 = x_6 \wedge x_3 = x_8 \wedge x_4 = x_{11} \wedge x_7 = x_{10} \wedge x_8 = x_9$
    - The discovered lemma:
      $\mathsf{dll}(x_1, x_2, x_3, x_4, n_1) \wedge n_1 \geq 2 \vdash \exists x_{5,...,11}, n_2.(\mathsf{dll}(x_5, x_6, x_7, x_8, n_2) *$
      $x_9 \mapsto x_{10}, x_{11} \wedge x_1 = x_5 \wedge x_2 = x_6 \wedge x_3 = x_8 \wedge x_4 = x_{11} \wedge x_7 = x_{10} \wedge x_8 = x_9)$
    - Normalize to obtain the motivating lemma L_3:
      $L_3 \triangleq \mathsf{dll}(a, b, c, d, m) \wedge m \geq 2 \vdash \exists w.(\mathsf{dll}(a, b, w, c, m-1) * c \mapsto w, d)$

Proof System: General Inference Rules

$$(\bot^1_{\mathsf{L}})\ \frac{}{\mathcal{H},\ \mathcal{L},\ F_1 * u \overset{\iota_1}{\mapsto} \vec{v} * u \overset{\iota_2}{\mapsto} \vec{t} \vdash F_2}$$

$$(\bot^2_{\mathsf{L}})\ \frac{}{\mathcal{H},\ \mathcal{L},\ F_1 \wedge \Pi_1 \vdash F_2}\ \Pi_1 \rightarrow \mathsf{false}$$

$$(\vdash_{\Pi})\ \frac{}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \vdash \Pi_2}\ \Pi_1 \rightarrow \Pi_2$$

$$(=_{\mathsf{L}})\ \frac{\mathcal{H},\ \mathcal{L},\ F_1[u/v] \vdash F_2[u/v]}{\mathcal{H},\ \mathcal{L},\ F_1 \wedge u{=}v \vdash F_2}$$

$$(\exists_{\mathsf{L}})\ \frac{\mathcal{H},\ \mathcal{L},\ F_1[u/x] \vdash F_2}{\mathcal{H},\ \mathcal{L},\ \exists x.F_1 \vdash F_2}\ u \notin \mathtt{FV}(F_2)$$

$$(\mathsf{CA})\ \frac{\mathcal{H},\ \mathcal{L},\ F_1 \wedge \Pi \vdash F_2 \qquad \mathcal{H},\ \mathcal{L},\ F_1 \wedge \neg\Pi \vdash F_2}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash F_2}$$

$$(\exists_{\mathsf{R}})\ \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.F_2[u/v]}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}, v.(F_2 \wedge u{=}v)}$$

$$(\mathsf{P}_{\mathsf{R}})\ \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(F_2 * F_i^{\mathsf{P}}(\vec{u}))}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(F_2 * \mathsf{P}(\vec{u}))}\ F_i^{\mathsf{P}}(\vec{u}) \overset{\mathrm{def}}{\Rightarrow} \mathsf{P}(\vec{u})$$

$$(\mathsf{E}_{\mathsf{L}})\ \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash F_2}{\mathcal{H},\ \mathcal{L},\ F_1 * \mathsf{emp} \vdash F_2}$$

$$(*\mapsto)\ \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(F_2 \wedge u{=}t \wedge \vec{v}{=}\vec{w})}{\mathcal{H},\ \mathcal{L},\ F_1 * u \overset{\iota}{\mapsto} \vec{v} \vdash \exists \vec{x}.(F_2 * t \overset{\iota}{\mapsto} \vec{w})}\ u, \vec{v} \# \vec{x}$$

$$(\mathsf{E}_{\mathsf{R}})\ \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.F_2}{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(F_2 * \mathsf{emp})}$$

$$(*\mathsf{P})\ \frac{\mathcal{H},\ \mathcal{L},\ F_1 \vdash \exists \vec{x}.(F_2 \wedge \vec{u}{=}\vec{v})}{\mathcal{H},\ \mathcal{L},\ F_1 * \mathsf{P}(\vec{u}) \vdash \exists \vec{x}.(F_2 * \mathsf{P}(\vec{v}))}\ \vec{u} \# \vec{x}$$

(More details are in our paper)

Proof System: Induction and Lemma Rules

$$(\mathsf{ID})\ \frac{\mathcal{H}',\ \mathcal{L},\ \Sigma_1 * F_1^{\mathsf{P}}(\vec{u}) \wedge \Pi_1 \vdash F_2 \qquad \ldots \qquad \mathcal{H}',\ \mathcal{L},\ \Sigma_1 * F_m^{\mathsf{P}}(\vec{u}) \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2}\ \mathsf{P}(\vec{u}) \stackrel{\mathsf{def}}{=} F_1^{\mathsf{P}}(\vec{u}) \vee \ldots \vee F_m^{\mathsf{P}}(\vec{u}); \dagger$$

$\dagger : \mathcal{H}' \triangleq \mathcal{H} \cup \{\, H \,\}$, where H is obtained by freshly renaming all variables of $\Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2$

$$(\mathsf{IH})\ \frac{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4\},\ \mathcal{L},\ F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2}{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \vdash F_4\},\ \mathcal{L},\ \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \vdash F_2}\ \begin{array}{l} \mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v}); \\ \exists \theta, \Sigma. (\Sigma_1 * \mathsf{P}(\vec{u}) \ \cong\ \Sigma_3\theta * \mathsf{P}(\vec{v})\theta * \Sigma) \wedge (\Pi_1 \rightarrow \Pi_3\theta) \end{array}$$

$$(\mathsf{LM_L})\ \frac{\mathcal{H},\ \mathcal{L} \cup \{\Sigma_3 \wedge \Pi_3 \vdash F_4\},\ F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2}{\mathcal{H},\ \mathcal{L} \cup \{\Sigma_3 \wedge \Pi_3 \vdash F_4\},\ \Sigma_1 \wedge \Pi_1 \vdash F_2}\ \exists \theta, \Sigma.\ (\Sigma_1 \cong \Sigma_3\theta * \Sigma) \wedge (\Pi_1 \rightarrow \Pi_3\theta)$$

$$(\mathsf{LM_R})\ \frac{\mathcal{H},\ \mathcal{L} \cup \{F_3 \vdash \exists \vec{w}.(\Sigma_4 \wedge \Pi_4)\},\ F_1 \vdash \exists \vec{x}.(F_3\theta * \Sigma \wedge \Pi_2)}{\mathcal{H},\ \mathcal{L} \cup \{F_3 \vdash \exists \vec{w}.(\Sigma_4 \wedge \Pi_4)\},\ F_1 \vdash \exists \vec{x}.(\Sigma_2 \wedge \Pi_2)}\ \exists \theta, \Sigma.\ (\Sigma_4\theta * \Sigma \cong \Sigma_2) \wedge (\vec{w}\theta \subseteq \vec{x})$$

Proof System: Synthesis Rules

$$(\mathsf{U}^1_\Pi)\ \frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \Pi_2\}}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \wedge U(\vec{x}) \vdash \Pi_2}$$

$$(\mathsf{U}^1_\Sigma)\ \frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \mathsf{false}\}}{\mathcal{H},\ \mathcal{L},\ \Sigma_1 \wedge \Pi_1 \wedge U(\vec{x}) \vdash \Pi_2}\ \Sigma_1 \neq \mathsf{emp}$$

$$(\mathsf{U}^2_\Pi)\ \frac{\mathcal{A} \triangleq \{\Pi_1 \rightarrow \exists \vec{w}.(\Pi_2 \wedge U(\vec{x}))\}}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \vdash \exists \vec{w}.(\Pi_2 \wedge U(\vec{x}))}$$

$$(\mathsf{U}^2_\Sigma)\ \frac{\mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow \mathsf{false}\}}{\mathcal{H},\ \mathcal{L},\ \Pi_1 \wedge U(\vec{x}) \vdash \exists \vec{w}.(\Sigma_2 \wedge \Pi_2)}\ \Sigma_2 \neq \mathsf{emp}$$

$$(\mathsf{U}_{\mathsf{IH}})\ \frac{\mathcal{H} \cup \{\, \Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \wedge U(\vec{y}) \vdash F_4 \,\},\ \mathcal{L},\ F_4\theta * \Sigma \wedge \Pi_1 \vdash F_2 \quad \mathcal{A} \triangleq \{\Pi_1 \wedge U(\vec{x}) \rightarrow (\Pi_3 \wedge U(\vec{y}))\theta\}}{\mathcal{H} \cup \{\Sigma_3 * \mathsf{P}(\vec{v}) \wedge \Pi_3 \wedge U(\vec{y}) \vdash F_4\},\ \mathcal{L},\ \Sigma_1 * \mathsf{P}(\vec{u}) \wedge \Pi_1 \wedge U(\vec{x}) \vdash F_2}\ \dagger$$

with $\dagger$: $\mathsf{P}(\vec{u}) \prec \mathsf{P}(\vec{v});\ \exists \theta, \Sigma.\, (\Sigma_1 * \mathsf{P}(\vec{u}) \cong \Sigma_3\theta * \mathsf{P}(\vec{v})\theta * \Sigma)$

Proof System: Proof Search Procedure

Procedure Prove$(\mathcal{H}, \mathcal{L}, F_1 \vdash F_2, mode)$

$\overline{\mathcal{R}} \leftarrow \{\, \mathsf{Unify}(\mathsf{R}, (\mathcal{H}, \mathcal{L}, F_1 \vdash F_2)) \mid \mathsf{R} \in \mathcal{R} \,\}$
$\mathcal{L}_{\text{syn}} \leftarrow \varnothing$
if $(mode = \textsc{SynLM})$ **and** NeedLemmas$(F_1 \vdash F_2, \overline{\mathcal{R}})$ **then**
  $\mathcal{L}_{\text{syn}} \leftarrow$ SynthesizeLemma$(\mathcal{L},\ F_1 \vdash F_2)$ *// Synthesize new lemmas*
  $\overline{\mathcal{R}} \leftarrow \overline{\mathcal{R}} \cup \{\, \mathsf{Unify}(\mathsf{R}, (\mathcal{H}, \mathcal{L}_{\text{syn}}, F_1 \vdash F_2)) \mid \mathsf{R} \in \{\mathsf{LM}_{\mathrm{L}}, \mathsf{LM}_{\mathrm{R}}\} \,\}$
for each $\bar{\mathsf{R}}$ **in** $\overline{\mathcal{R}}$ **do**
  $\mathcal{A} \leftarrow \varnothing$
  if IsSynthesisRule$(\bar{\mathsf{R}})$ **then** *// $\bar{\mathsf{R}} \in \{\, \mathsf{U}^1_{\Pi}, \mathsf{U}^2_{\Pi}, \mathsf{U}^1_{\Sigma}, \mathsf{U}^2_{\Sigma}, \mathsf{U}_{IH} \,\}$*
    $\mathcal{A} \leftarrow$ Assumptions$(\bar{\mathsf{R}})$
  if IsAxiomRule$(\bar{\mathsf{R}})$ **then**
    $\xi \leftarrow$ CreateWitnessProofTree$(F_1 \vdash F_2, \bar{\mathsf{R}}, \varnothing)$
    return $(\textsc{Valid}\langle\xi\rangle, \mathcal{L}_{\text{syn}}, \mathcal{A})$
  $(\mathcal{H}_i, \mathcal{L}_i, F_{1i} \vdash F_{2i})_{i=1\ldots n} \leftarrow$ Premises$(\bar{\mathsf{R}})$
  $(r_i, \mathcal{L}^i_{\text{syn}}, \mathcal{A}_i)_{i=1\ldots n} \leftarrow$ Prove$(\mathcal{H}_i,\ \mathcal{L} \cup \mathcal{L}_{\text{syn}} \cup \mathcal{L}_i,\ F_{1i} \vdash F_{2i}, mode)_{i=1\ldots n}$
  if $r_i = \textsc{Valid}\langle\xi_i\rangle$ **for all** $i = 1 \ldots n$ **then**
    $\xi \leftarrow$ CreateWitnessProofTree$(F_1 \vdash F_2, \bar{\mathsf{R}}, \{\, \xi_1, \ldots, \xi_n \,\})$
    return $(\textsc{Valid}\langle\xi\rangle,\ \mathcal{L}_{\text{syn}} \cup \mathcal{L}^1_{\text{syn}} \cup \ldots \cup \mathcal{L}^n_{\text{syn}},\ \mathcal{A} \cup \mathcal{A}_1 \cup \ldots \cup \mathcal{A}_n)$
return $(\textsc{Unkn}, \varnothing, \varnothing)$

Implementation and Experiment

- Implementation:
  - $\textsc{Songbird}[\mathrm{LS}] = \textsc{Songbird} + \mathrm{Lemma\ Synthesis}$.
  - Built on top of our existing prover $\textsc{Songbird}$ [Ta et al., FM'2016], written in OCaml.
  - https://songbird-prover.github.io/lemma-synthesis

Implementation and Experiment

- Implementation:
  - $\textsc{Songbird}[\mathrm{LS}] = \textsc{Songbird} + \mathrm{Lemma\ Synthesis}$.
  - Built on top of our existing prover $\textsc{Songbird}$ [Ta et al., FM'2016], written in OCaml.
  - https://songbird-prover.github.io/lemma-synthesis

- Experiment:
  - Literature benchmarks: **sll_entl**, **slrd_entl** (SL-COMP-2014) and **slrd_indt** [Ta et al., FM'2016].
  - New benchmark with more complex entailments: **slrd_lm**.
  - Testing environment: Ubuntu server 14.04, CPU Intel E5-2620 (2.4GHz), RAM 64GB.

Experiment: Literature Benchmarks

	Benchmark		Proved Entailments						Average Time (s)						Lemma Syn. and App.	
	Category		Sld	Slk	Spn	Ccl	Sbd	Sls	Sld	Slk	Spn	Ccl	Sbd	Sls	#Lemma	Avg Time
sll_entl	bolognesa	57	0	0	**57**	0	**57**	**57**	–	–	0.41	–	2.46	0.32	0	–
	clones	60	0	**60**	**60**	**60**	**60**	**60**	–	0.06	0.06	0.01	0.07	0.01	0	–
	smallfoot	54	0	**54**	**54**	**54**	**54**	**54**	–	0.05	0.05	0.22	0.06	0.08	0	–
slrd_entl	singly-ll	64	12	48	3	63	**64**	**64**	0.08	0.13	0.04	0.03	0.13	0.03	0	–
	doubly-ll	37	14	17	9	29	25	**35**	2.74	0.18	0.04	3.88	0.45	2.63	18[10]	2.8
	nested-ll	11	0	5	**11**	7	**11**	**11**	–	0.44	0.04	2.38	0.21	0.04	0	–
	skip-list	13	0	4	**13**	5	**13**	**13**	–	0.27	0.08	0.11	0.63	0.10	0	–
	tree	26	12	14	0	23	23	**24**	9.16	0.21	–	2.55	0.50	0.09	0	–
slrd_indt	ll/ll2	24	0	0	0	**24**	**24**	**24**	–	–	–	2.51	0.50	3.27	14[10]	0.9
	ll-even/odd	20	0	0	0	**20**	**20**	**20**	–	–	–	1.73	2.52	0.07	0	–
	ll-left/right	20	0	0	0	**20**	**20**	**20**	–	–	–	0.97	0.53	0.06	0	–
	misc.	32	0	0	0	31	**32**	**32**	–	–	–	8.20	1.72	0.61	2[2]	0.7
Total		418	38	202	207	336	403	**414**†	3.93	0.11	0.15	1.70	0.79	0.54	34[22]‡	1.9

Sld: SLIDE[Iosif et al., CADE'2013] **Spn**: SPEN[Enea et al., APLAS'2014] **Ccl**: CYCLIST[Brotherston e
Slk: SLEEK[Chin et al., SCP'2012] **Sbd**: SONGBIRD[Ta et al., FM'2016] **Sls**: SONGBIRD[LS][This Wo
Benchmarks: **sll_entl**, **slrd_entl**: SL-COMP 2014, **slrd_indt**: [Ta et al., FM'2016]
Sls proves 414†/418=99.04% benchmarks, synthesizes 34‡/418=0.08 lemmas per entailment

Experiment: New Benchmark slrd_lm (Manually Created)

Category		Proved Entails			Avg. Time (s)			Lemma Synthesis		Lemma Application		
		Ccl	Sbd	Sls	Ccl	Sbd	Sls	#Lemma	A.Time	#Convert	#Split	#Combine
ll	20	17	18	**20**	26.27	4.84	10.25	28 [20]	0.3	18 (25)	0	2 (2)
ll-even/odd	17	**17**	**17**	**17**	0.21	0.03	0.02	0	–	0	0	0
ll+arith	72	0	10	**72**	–	0.04	2.82	81 [81]	0.6	0	81 (143)	0
ll-sorted	25	0	3	**19**	–	0.03	0.46	14 [14]	0.3	0	0	14 (30)
dll	22	**22**	**22**	**22**	0.38	0.80	6.81	12 [7]	4.9	7 (8)	0	0
dll+arith	94	0	17	**94**	–	1.63	26.39	236 [158]	8.1	74 (74)	69 (103)	15 (15)
ll/dll	5	**5**	**5**	**5**	0.07	0.03	0.03	0	–	0	0	0
ll/dll+arith	15	0	5	**15**	–	0.08	21.89	12 [10]	18.9	8 (8)	2 (2)	0
tree	18	7	7	**18**	0.18	0.04	7.50	18 [12]	0.2	8 (11)	0	4 (5)
tree+arith	12	0	3	**12**	–	1.15	42.81	20 [16]	10.6	8 (8)	4 (5)	4 (4)
Total	300	68	107	**294**†	6.76	1.29	13.69	421 [318]‡	5.9	123 (134)	156 (253)	39 (56)

$\dagger$: SONGBIRD[LS] can prove $294/300 = 98\%$ of the benchmark

$\ddagger$: $421/300 = 1.40$ lemmas synthesized per entailment

$318/300 = 1.06$ lemmas applied per entailment

Conclusion & Future Works

- Problem: proving sophisticated separation logic entailments containing inductive heap predicates and linear arithmetic.

Conclusion & Future Works

- Problem: proving sophisticated separation logic entailments containing inductive heap predicates and linear arithmetic.
- Solution:
  - Propose a framework to automatically synthesize auxiliary lemmas to assist in proving the entailments:
    + Lemmas are synthesized by *templates* + *induction proof* + *constraint solving*.
  - Integrate the framework into the prover SONGBIRD[LS]: https://songbird-prover.github.io/lemma-synthesis

Conclusion & Future Works

- Problem: proving sophisticated separation logic entailments containing inductive heap predicates and linear arithmetic.
- Solution:
  - Propose a framework to automatically synthesize auxiliary lemmas to assist in proving the entailments:
    + Lemmas are synthesized by *templates* + *induction proof* + *constraint solving*.
  - Integrate the framework into the prover SONGBIRD[LS]: https://songbird-prover.github.io/lemma-synthesis
- Future works:
  - Support more expressive logic fragments: containing non-linear arithmetic and other constraints like set, multiset.
  - Apply into practical program verification: integrating into the HIP/SLEEK system [Chin et al., SCP'2012].

References i

Berdine, J., Calcagno, C., and O'Hearn, P. W. (2004).
A Decidable Fragment of Separation Logic.
In *International Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS)*, pages 97–109.

Brotherston, J., Distefano, D., and Petersen, R. L. (2011).
Automated Cyclic Entailment Proofs in Separation Logic.
In *International Conference on Automated Deduction (CADE)*, pages 131–146.

Bundy, A. (2001).
The automation of proof by mathematical induction.
In *Handbook of Automated Reasoning (in 2 volumes)*, pages 845–911.

Chin, W.-N., David, C., Nguyen, H. H., and Qin, S. (2012).
Automated verification of shape, size and bag properties via user-defined predicates in Separation Logic.
Science of Computer Programming (SCP), 77(9):1006–1036.

References ii

Chu, D.-H., Jaffar, J., and Trinh, M.-T. (2015).
Automatic induction proofs of data-structures in imperative programs.
In *Conference on Programming Language Design and Implementation (PLDI)*, pages 457–466.

Cook, B., Haase, C., Ouaknine, J., Parkinson, M. J., and Worrell, J. (2011).
Tractable Reasoning in a Fragment of Separation Logic.
In *International Conference on Concurrency Theory (CONCUR)*, pages 235–249.

Enea, C., Lengál, O., Sighireanu, M., and Vojnar, T. (2014).
Compositional Entailment Checking for a Fragment of Separation Logic.
In *Asian Symposium on Programming Languages and Systems (APLAS)*, pages 314–333.

Iosif, R., Rogalewicz, A., and Simácek, J. (2013).
The Tree Width of Separation Logic with Recursive Definitions.
In *International Conference on Automated Deduction (CADE)*, pages 21–38.

References iii

Pérez, J. A. N. and Rybalchenko, A. (2011).
Separation Logic + Superposition Calculus = Heap Theorem Prover.
In *Conference on Programming Language Design and Implementation (PLDI)*, pages 556–566.

Piskac, R., Wies, T., and Zufferey, D. (2013).
Automating Separation Logic Using SMT.
In *International Conference on Computer Aided Verification (CAV)*, pages 773–789.

Schrijver, A. (1986).
Theory of Linear and Integer Programming.
John Wiley & Sons, Inc., New York, NY, USA.

Ta, Q., Le, T. C., Khoo, S., and Chin, W. (2016).
Automated mutual explicit induction proof in separation logic.
In *FM 2016: Formal Methods - 21st International Symposium, Limassol, Cyprus, November 9-11, 2016, Proceedings*, pages 659–676.

Take away

- Proving separation logic entailments with inductive heap predicates and linear arithmetic.
- Synthesizing auxiliary lemmas by *templates* + *induction proof* + *constraints solving* to assist in the entailment proofs.
- Separation Logic prover SONGBIRD[LS]: https://songbird-prover.github.io/lemma-synthesis

Take away

- Proving separation logic entailments with inductive heap predicates and linear arithmetic.
- Synthesizing auxiliary lemmas by *templates* + *induction proof* + *constraints solving* to assist in the entailment proofs.
- Separation Logic prover SONGBIRD[LS]: https://songbird-prover.github.io/lemma-synthesis

Thank you!